Practical : 8	Implementation of Graph and Searching (DFS and BFS)
Date : 25/09/2026	

Aim: To implement a graph using an adjacency list and to perform **Breadth First Search (BFS)** and **Depth First Search (DFS)** traversals on it, each as a separate program.

Objective :- To implement a graph in C++, traverse it using BFS and DFS, and to analyze and compare the time and space complexities of both searching methods.

A. Graph Representation (Adjacency List)

1. Start.
2. Read the number of vertices V and the number of edges E.
3. Create an array of V lists, one list for each vertex.
4. For each edge (u, v), add v to the list of u and u to the list of v (undirected graph).
5. Display the adjacency list.
6. Stop.

B. BFS Algorithm

1. Start.
2. Read the starting vertex s.
3. Create a visited array (all false) and an empty queue.
4. Mark s as visited and push s into the queue.
5. While the queue is not empty, remove the front vertex u and print it.
6. For every neighbour v of u that is not visited, mark v as visited and push it into the queue.
7. Stop when the queue becomes empty.

C. DFS Algorithm

1. Start.
2. Read the starting vertex s.
3. Create a visited array (all false).
4. Call DFS(s): mark s as visited and print s.
5. For every neighbour v of s that is not visited, call DFS(v).
6. Return when all neighbours are visited (backtrack).
7. Stop.

Program A: BFS (practical8_bfs.cpp)

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
class Graph {
    int V;
    vector<vector<int>> adj;
public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }
    // Add an undirected edge
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    // Display adjacency list
    void displayGraph() {
        cout << "\n--- Adjacency List ---" << endl;
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (int v : adj[i]) {
                cout << v << " ";
            }
            cout << endl;
        }
    }
    // Breadth First Search (uses Queue)
    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);
        cout << "\n--- BFS Traversal ---" << endl;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            cout << u << " ";
            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        cout << endl;
    }
};
int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;
    Graph g(V);
    cout << "Enter edges (u v):" << endl;
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        if (u < 0 || u >= V || v < 0 || v >= V) {
            cout << "Invalid edge. Vertices must be between 0 and "
                 << V - 1 << "." << endl;
            i--;
            continue;
        }
        g.addEdge(u, v);
    }
```

```
    int start;
    cout << "Enter starting vertex: ";
    cin >> start;
    if (start < 0 || start >= V) {
        cout << "Invalid starting vertex." << endl;
        return 0;
    }
    g.displayGraph();
    g.BFS(start);
    return 0;
}
```

Output:

Sample graph used: 6 vertices (0 to 5) and 7 edges: 0-1, 0-2, 1-3, 1-4, 2-4, 3-5, 4-5.

Output

```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
3 5
4 5
Enter starting vertex: 0

--- Adjacency List ---
0 -> 1 2
1 -> 0 3 4
2 -> 0 4
3 -> 1 5
4 -> 1 2 5
5 -> 3 4

--- BFS Traversal ---
0 1 2 3 4 5


=== Code Execution Successful ===
```

Program B: DFS (practical8_dfs.cpp)

```
#include <iostream>
#include <vector>
using namespace std;
class Graph {
    int V;
    vector<vector<int>> adj;
public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }
    // Add an undirected edge
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    // Display adjacency list
    void displayGraph() {
        cout << "\n--- Adjacency List ---" << endl;
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (int v : adj[i]) {
                cout << v << " ";
            }
            cout << endl;
        }
    }
    // Depth First Search (recursive helper)
    void DFSUtil(int u, vector<bool> &visited) {
        visited[u] = true;
        cout << u << " ";
        for (int v : adj[u]) {
            if (!visited[v]) {
                DFSUtil(v, visited);
            }
        }
    }
    void DFS(int start) {
        vector<bool> visited(V, false);
        cout << "\n--- DFS Traversal ---" << endl;
        DFSUtil(start, visited);
        cout << endl;
    }
};
int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;
    Graph g(V);
    cout << "Enter edges (u v):" << endl;
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        if (u < 0 || u >= V || v < 0 || v >= V) {
            cout << "Invalid edge. Vertices must be between 0 and "
                 << V - 1 << "." << endl;
            i--;
            continue;
        }
        g.addEdge(u, v);
    }
    int start;
    cout << "Enter starting vertex: ";
    cin >> start;
    if (start < 0 || start >= V) {
        cout << "Invalid starting vertex." << endl;
```

```
            return 0;
        }
        g.displayGraph();
        g.DFS(start);
        return 0;
    }
```

Output:

Sample graph used: 6 vertices (0 to 5) and 7 edges: 0-1, 0-2, 1-3, 1-4, 2-4, 3-5, 4-5.

Output

```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
3 5
4 5
Enter starting vertex: 0

--- Adjacency List ---
0 -> 1 2
1 -> 0 3 4
2 -> 0 4
3 -> 1 5
4 -> 1 2 5
5 -> 3 4

--- DFS Traversal ---
0 1 3 5 4 2


=== Code Execution Successful ===
```

Analysis:

Time Complexity Analysis

Let V be the number of vertices and E the number of edges of the graph.

BFS:

The queue handles every vertex exactly once, so push and pop operations take O(V) in total.

Every adjacency list is scanned exactly once, so each edge is examined a constant number of times, giving O(E).

Time Complexity = O(V + E)

DFS:

DFSUtil is called exactly once for every vertex, because already visited vertices are skipped.

The loop inside each call scans the adjacency list of that vertex, so all calls together scan O(E) entries.

Time Complexity = O(V + E)

Note: if an adjacency matrix is used instead of an adjacency list, both traversals take $O(V^2)$ time, because finding the neighbours of a vertex needs a scan of a full row.

Space Complexity Analysis

BFS:

The visited array uses O(V) space.

The queue can hold a whole level of the graph at once. In the worst case (for example a star graph) this is O(V).

Space Complexity = O(V)

DFS:

The visited array uses O(V) space.

The recursion call stack holds one frame per vertex on the current path. In the worst case (a path-like graph) the depth is V.

Space Complexity = O(V)

The adjacency list itself needs O(V + E) space to store the graph.

Comparison of BFS and DFS

Feature	BFS	DFS
Data structure used	Queue (FIFO)	Stack / recursion (LIFO)
Traversal order	Level by level	Goes deep along a path, then backtracks
Time complexity	O(V + E)	O(V + E)
Space complexity	O(V)	O(V)
Shortest path (unweighted)	Yes	No
Typical uses	Shortest path, level order, nearest neighbours	Cycle detection, topological sort, connected components

Conclusion :-

Both BFS and DFS visit every vertex and edge of a graph in linear time O(V + E) when the graph is stored as an adjacency list, but they explore it in different ways:

1. The **BFS** method explores the graph level by level using a queue. It finds the shortest path in an unweighted graph, but the queue may need a lot of memory for wide graphs.

2. The **DFS** method goes as deep as possible before backtracking, using recursion or a stack. It is simple to implement and suits cycle detection and topological sorting. However, on very deep graphs the recursion can cause a **stack overflow** crash.